

Proje 3: BFS, IDS ve UCS

2021221054 - MUHAMMED EMİN KORKUNÇ

Problem Tanımı: Bir harita üzerinde şehirler arası yolların maliyetlerini ve bağlantılarını içeren bir grafik var. Amacımız başlangıç şehriden hedef şehre en kısa yolu bulmak. Yolların maliyeti, her iki şehir arasındaki mesafeyi temsil ediyor.

Şehirler: A, B, C, D, E, F

Bağlantılar ve maliyetler:

A → B: 1

A → C: 4

B → D: 2

B → E: 5

C → F: 3

D → E: 1

E → F: 1

1. BFS (Breadth-First Search)

BFS yalnızca seviye sırasına göre çalışır ve bu nedenle bir çözüm bulur ancak en düşük maliyetli yolu garanti etmez.

code

```
from collections import deque

def bfs(graph, start, goal):
    queue = deque([(start, [start])]) # (node, path)
    while queue:
        node, path = queue.popleft()
        if node == goal:
            return path
        for neighbor, _ in graph[node]:
            if neighbor not in path:
                queue.append((neighbor, path + [neighbor]))
    return None

# Kullanım
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('F', 3)],
    'D': [('E', 1)],
```

```

'E': [('F', 1)],
'F': []
}
start, goal = 'A', 'F'
bfs_result = bfs(graph, start, goal)
print("BFS Result:", bfs_result)

```

execute code

```

BFS Result: ['A', 'C', 'F']
(base) muhammedeminkorkunc@Desktop

```

örnek çalışma modeli step-by-step

Başlangıç: ['A']

İlk seviyede B ve C keşfedilir: ['A', 'B', 'C']

C üzerinden F'ye ulaşılır: ['A', 'C', 'F']

2. IDS (Iterative Deepening Search)

IDS, derinliği artırarak hedefe ulaşır. Maliyet hesaba katılmadığı için sonuç BFS ile aynıdır.

code

```

def dfs_limited(graph, node, goal, depth, path):
    if depth == 0 and node == goal:
        return path
    if depth > 0:
        for neighbor, _ in graph[node]:
            if neighbor not in path:
                result = dfs_limited(graph, neighbor, goal, depth - 1, path +
[neighbor])
                if result:
                    return result
    return None

def ids(graph, start, goal):
    depth = 0
    while True:
        result = dfs_limited(graph, start, goal, depth, [start])
        if result:
            return result
        depth += 1

graph = {
    'A': [('B', 1), ('C', 4)],

```

```

'B': [('D', 2), ('E', 5)],
'C': [('F', 3)],
'D': [('E', 1)],
'E': [('F', 1)],
'F': []
}
start, goal = 'A', 'F'
ids_result = ids(graph, start, goal)
print("IDS Result:", ids_result)

```

execute code

```

IDS Result: ['A', 'C', 'F']
(base) muhammedeminkorkunc@

```

örnek çalışma modeli step-by-step

Derinlik sınırı 0: Sadece A kontrol edilir, sonuç bulunmaz.

Derinlik sınırı 1: B ve C kontrol edilir, sonuç bulunmaz.

Derinlik sınırı 2: F keşfedilir ve sonuç: ['A', 'C', 'F']

3. UCS (Uniform-Cost Search)

UCS, en düşük maliyetli yolu garanti eder. Bu nedenle, A → B → D → E → F yolunu seçti.

code

```

import heapq
def ucs(graph, start, goal):
    priority_queue = [(0, start, [start])] # (cost, node, path)
    visited = set()
    while priority_queue:
        cost, node, path = heapq.heappop(priority_queue)
        if node in visited:
            continue
        visited.add(node)
        if node == goal:
            return path, cost
        for neighbor, edge_cost in graph[node]:
            if neighbor not in visited:
                heapq.heappush(priority_queue, (cost + edge_cost, neighbor, path + [neighbor]))
    return None

```

```
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('F', 3)],
    'D': [('E', 1)],
    'E': [('F', 1)],
    'F': []
}

start, goal = 'A', 'F'

ucs_result, ucs_cost = ucs(graph, start, goal)
print("UCS Result:", ucs_result, "Cost:", ucs_cost)
```

execute code

```
UCS Result: ['A', 'B', 'D', 'E', 'F'] Cost: 5
(base) muhammedemin.korkunc@Desktop: python 8
```

örnek çalışma modeli step-by-step

Başlangıç: (A, maliyet=0)

Düşük maliyetli yol olarak B seçilir (A → B, maliyet=1).

D üzerinden E'ye, ardından F'ye ulaşılır: ['A', 'B', 'D', 'E', 'F'],

toplam maliyet=5